
SOFTWARE ENGINEERING

NOTEBOOK

PRODUCED BY: *vaeVictis*

leo.expat@gmail.com

SEE SECTION ?? FOR SOCIAL CONTACTS.

THIS DOCUMENT IS WRITTEN IN L^AT_EX

ENJOY!

Table of Contents

1	Brief Introduction to Latex	2
1.1	Downloading Tex Studio	2
1.2	Source Code and Resume Templates	2
1.3	LaTeX Files	2
1.3.1	LaTeX Commands	3
1.4	General Items	3
1.5	Terminology to Know	3
1.6	Tex Studio Commands	3
1.7	Creating Templates	3
2	Solid Principles	5
2.1	Single-Responsibility	5
2.2	Open-Close	5
2.3	Liskov Substitution	5
2.4	Interface Segregation	6
2.5	Dependency Inversion	6
3	Design Patterns	7
3.1	Creational Patterns	7
3.1.1	Factory Method	7
3.1.2	Abstract Factory	9
3.1.3	Builder	9
3.1.4	Prototype	10
3.1.5	Singleton	11
3.2	Behavioral Patterns	11
3.2.1	Chain of Responsibility	11
3.2.2	Command	12
3.2.3	Iterator	12
3.2.4	Mediator	13
3.2.5	Memento	13
3.2.6	Observer	13
3.2.7	State	14
3.2.8	Strategy	14
3.2.9	Template Method	15
3.2.10	Visitor	15
3.3	Structural Patterns	16
3.3.1	Adapter	16
3.3.2	Bridge	16

3.3.3	Composite	17
3.3.4	Decorator	17
3.3.5	Facade	18
3.3.6	Flyweight	18
3.3.7	Proxy	18
4	References	21

About This File

This file was created for the benefit of all teachers and students wanting to use Latex for tests/exams/lessons/thesis/articles etc.

The entirety of the contents within this file, and folder, are free for public use.

Brief Introduction to Latex

LaTeX, which is pronounced «Lah-tech» or «Lay-tech», is a document preparation system for high-quality typesetting. It is most often used for medium-to-large technical or scientific documents but it can be used for almost any form of publishing.

When using Latex, ***make sure to keep your source code organized by indenting and using sections/chapters/subsections.*** Not keeping your source code organized makes it harder to fix possible errors in your code.

1.1 Downloading Tex Studio

Latex is free to download and is available on all types of operating systems. To download $\text{\LaTeX}$, click the url to the right: <https://www.texstudio.org/>

1.2 Source Code and Resume Templates

If interested in obtaining the source code for this guide, see:

<https://sourceforge.net/p/latex-source-code/wiki/Download/>

If interested in obtaining some of the resume templates I have created, see:

<https://sourceforge.net/p/latex-resume-template/wiki/download/>

1.3 LaTeX Files

LaTeX files have names that end with the extension .tex: for example, an acceptable file name might be myfile.tex. (Never use spaces in file names.) The input file contains both the text of

your document and the LATEX commands needed to format it. The first command in the file, `\documentclass`, defines the style of the document.

1.3.1 LaTeX Commands

To distinguish them from text, all LATEX commands (also called control sequences) start with a backslash `\`. A command name consists of letters only and is ended by a space or a non letter.

1.4 General Items

`\usepackage`

Packages in latex allow for the user to include cool and unique features into ones document. Most packages for latex will be found on www.ctan.org

```
\usepackage{package}
```

1.5 Terminology to Know

- Preamble: All the code that precedes your `\begin{document}`. The preamble section of your document is where all the formatting takes place.
- Commands: Commands are special words that determine $\text{\LaTeX}$ behavior.
- Environments: Environments are used to format blocks of text in a $\text{\LaTeX}$ documents

1.6 Tex Studio Commands

- The button with the magnifying glass, labeled view, in the top part of Tex Studio allows you to view your document sidebyside with your code.
- The green arrows next to the magnifying class, labeled compile, allows you to update your document with any new code you include. Compiling also allows you to check if there are any errors in your code.

1.7 Creating Templates

One of the many great things about $\text{\LaTeX}$ is that offers many ways to avoid some of the repetitive things that come with creating several documents. One of the features $\text{\LaTeX}$ offers is to create personal user templates for files that a user often uses but does not want to continuously type out every time.

For example, lets say you wanted to save a general layout of a lesson document that included **both** the preamble and some of the actual text and commands in the document. To create the template, one would first proceed to the folder originally containing this pdf (The Contents You Downloaded from the Zip File)) and then go to the folder labeled **Files to make Templates out of**. Once there, click the file named: **Lesson Template**. Once you have opened that file in texstudio, go to **File**, then **Make Template**. Once that is done, give the template a name and then press ok.

After creating a template, all a user needs to do to use that template is to go back to **File**, and then **New From Template** and then click the template you created.

Solid Principles

SOLID is a mnemonic acronym for five design principles intended to make object-oriented designs more understandable, flexible, and maintainable.

The principles are a subset of many principles promoted by American software engineer and instructor **Robert C. Martin**, [1][2][3].

They were first introduced in his 2000 paper Design Principles and Design Patterns, while the acronym was introduced later, around 2004, by **Michael Feathers**.

Although the SOLID principles apply to any object-oriented design, they can also form a core philosophy for methodologies such as agile development or adaptive software development.

2.1 Single-Responsibility

"There should never be more than one reason for a class to change." [5]

In other words, every class should have only one responsibility. [6]

2.2 Open-Close

"Software entities ... should be open for extension, but closed for modification." [7]

2.3 Liskov Substitution

"Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it." [8]

See also: https://en.wikipedia.org/wiki/Design_by_contract

2.4 Interface Segregation

"Clients should not be forced to depend upon interfaces that they do not use." [\[9\]](#) [\[4\]](#)

2.5 Dependency Inversion

"Depend upon abstractions, [not] concretions." [\[10\]](#) [\[4\]](#)

Design Patterns

Design Patterns describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.

3.1 Creational Patterns

3.1.1 Factory Method

Factory Method uses factory methods to deal with the problem of creating objects without having to specify the exact class of the object that will be created.

This is done by creating objects by calling a factory method — either specified in an interface and implemented by child classes, or implemented in a base class and optionally overridden by derived classes — rather than by calling a constructor.

Intent The object's creation may lead to a significant duplication of code, may require information not accessible to the composing object, may not provide a sufficient level of abstraction, or may otherwise not be part of the composing object's concerns.

The factory method design pattern handles these problems by defining a separate method for creating the objects, which subclasses can then override to specify the derived type of product that will be created.

When to use this pattern

- the exact types and dependencies of the objects your code should work with are not known

- there is the need to provide users of a library or framework with a way to extend its internal components
- there is the concern to save system resources by reusing existing objects instead of re-building them each time.

Structure



Figure 1: *Factory Method Class Diagram*

The structure of the Factory Method is quite easy

- **Product** defines the interface common to all the objects.
- **Concrete Products** are the different implementations of the previous interface.
- **Creator** is the base class which declares the factory method.
 - **factory method** should either be abstract or return a default product.
 - **returned type** must match the interface

Pros and Cons

Relation with other Design Patterns

3.1.2 Abstract Factory

Abstract Factory makes easier to produce families of related items without specifying their concrete classes. This is achieved declaring explicit interfaces for each distinct product of the product family and making all variants of products need to follow those interfaces.

An Abstract Factory is finally declared with a list of creation methods for all the items that are part of the product family. For each variant of a product family, a separate factory class based on the AbstractFactory interface will be needed and clients access with both factories and products via their abstract interfaces.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.1.3 Builder

Builder facilitates the construction of complex objects step by step, producing different types and representations of an object using the same construction code. This is achieved extracting the construction code out of its own class and moving it to a separate Builder class. Construction is managed with a set of steps executed in series and only in case they are necessary for producing a specific configuration of the object. A specific Builder class could be implemented to manage the same building steps in a different manner. A Director class, not mandatory, can be implemented to collect the steps in a specific order, while the Builder provides the implementation of what to do.

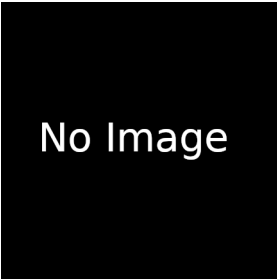
Intent

When to use this pattern

-
-
-

Structure

Pros and Cons



No Image

Relation with other Design Patterns

3.1.4 Prototype

Prototype enables the copy of existing objects without implementing code which depends on their classes. This is achieved delegating the cloning process to the actual objects that are being cloned. A common interface is implemented for all the objects that support cloning. The clone method is similar for all classes: it creates an object of the current class and carries over all the fields values of the old cloned one. Copying private fields is possible for any programming language which allows to access private fields of other objects of the same class. An object which could be cloned is called Prototype. Cloning is an alternative to subclassing for objects with many fields and configurations.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.1.5 Singleton

Singleton ensures that a class could have one instance only, while providing a global access point to this instance. This functionality could be interpreted as a violation of the Single Responsibility Principle.

The implementation of the Singleton has 2 steps in common:

- default constructor is made private, to prevent other objects from calling it
- a static creation method acts as a constructor, calling the private constructor to create an object and save it in a static field. Following calls to this method will return the cached object.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2 Behavioral Patterns

3.2.1 Chain of Responsibility

Chain of Responsibility makes possible to pass a request along a chain of handlers, each of which decides upon receiving a request either to process the request or to pass it to the next handler in the chain.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.2 Command

Command

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.3 Iterator

Iterator enables the client to traverse elements of a collection without exposing its underlying representation. This is achieved extracting the traversal behavior of a collection into a separate object, called iterator. Details of the traversals are encapsulated into the iterator class; in this way several iterators can go through the same collection at the same time and independently of each other. Iterators should implement the same interface in order to make the client code compatible with any collection type or any traversal algorithm, as long as there's a proper iterator.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.4 Mediator

Mediator reduces chaotic and confusing dependencies between objects, restricting direct communications between the objects and forcing them to collaborate only through a mediator object. This is achieved cutting all direct communication between the components which are required to be independent of each other and ...

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.5 Memento

Memento

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.6 Observer

Observer

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.7 State

State

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.8 Strategy

Strategy

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.9 Template Method

Template Method

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.2.10 Visitor

Visitor

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.3 Structural Patterns

3.3.1 Adapter

Adapter allows the collaboration among objects with incompatible interfaces. This is achieved through the implementation of an Adapter which make one interface understandable by an object of a different class. The adapter gets an interface compatible with one existing object. Through this interface, the objects can call the adapter's methods. Upon receiving a call, the adapter passes the request to the second object, in a format and order it could understand. An Adapter could work in both directions.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.3.2 Bridge

Bridge splits a large class or a set of closely related classes into two separate hierarchies which can be developed independently of each other. This is achieved by switching from inheritance to object composition. One of the dimensions is extracted into a separate class hierarchy, therefore the original classes will reference an object of the new hierarchy instead of having all its states and behaviors coded within the class. The original class implement a reference field for each "extracted-dimension" classes This extraction makes a class able to delegate any 'dimension-related' work to the linked 'dimension-related' object.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.3.3 Composite

Composite after enabling composition of objects into tree structures, this pattern let clients work with these structures as they were individual objects. This is achieved extending the class

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.3.4 Decorator

Decorator attaches new behaviors to objects by placing these objects inside special wrapper objects containing the desired behavior.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.3.5 Facade

Facade consists of a simplified interface to a complex subsystem which contains lots of moving parts. Functionalities provided by the Facade could be reduced to the subset the client is interested in.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.3.6 Flyweight

Flyweight reduces RAM consumption by sharing common parts of state between multiple objects, instead of creating copies of same data in each object.

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

3.3.7 Proxy

Proxy

Intent

When to use this pattern

-
-
-

Structure

Pros and Cons

Relation with other Design Patterns

References

- [1] Robert C. Martin (2014). *Principles of OOD*
Retrieved from: <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>
- [2] Robert C. Martin (2009). *Getting a SOLID start*
Retrieved from: <https://web.archive.org/web/20130917122741/https://sites.google.com/site/unclebobconsultingllc/getting-a-solid-start>
- [3] Sandu Metz (2009). *SOLID Object-Oriented Design*
Retrieved from:
<https://ghostarchive.org/varchive/v-2yFMzxqwU>
- [4] Robert C. Martin (2000). *Design Principles and Design Patterns*
Retrieved from:
https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf
- [5] Robert C. Martin (2015) *Single Responsibility Principle*
Retrieved from:
<https://web.archive.org/web/20150202200348/http://www.objectmentor.com/resources/articles/srp.pdf>
- [6] Robert C. Martin (2003) *Agile Software Development, Principles, Patterns, and Practices*
Retrieved from:
<https://web.archive.org/web/20150202200348/http://www.objectmentor.com/resources/articles/srp.pdf>

- [7] *Open/Closed Principle*
Retrieved from:
<https://web.archive.org/web/20150905081105/http://www.objectmentor.com/resources/articles/ocp.pdf>
- [8] *Liskov Substitution Principle*
Retrieved from:
<https://web.archive.org/web/20150905081111/http://www.objectmentor.com/resources/articles/lsp.pdf>
- [9] *Interface Segregation Principle*
Retrieved from:
<https://web.archive.org/web/20150905081110/http://www.objectmentor.com/resources/articles/isp.pdf>
- [10] *Dependency Inversion Principle*
Retrieved from:
<https://web.archive.org/web/20150905081103/http://www.objectmentor.com/resources/articles/dip.pdf>
- [11] *Clean Architecture: A Craftman's Guide to Software Structure and Design*
Retrieved from:
<https://books.google.com/books?id=uGE1DwAAQBAJ&q=2004+or+thereabouts+by+Michael+Feathers>
- [12]
Retrieved from: